

Function Basics

Functions save a block of code to be reused. Code inside `{ }` does **not** run until called.

```
function sayHello() {  
  console.log('Hello');  
}
```

Creates a reusable, named block of code.

Function Syntax

Syntax = rules for writing JavaScript functions.

```
function  sayHello()  { ... }
```

↓ ↓ ↓

Keyword **Name** **Body**

(creates it) (labels it) (code runs here)

Calling a Function

```
sayHello();
```

Runs the body. Can be called as many times as needed.

Parameters & Arguments

Pass data **into** a function. Parameters = placeholders. Arguments = actual values.

```
function calcTax(cost) { }  
  
calcTax(2000);  
  
function greet(name, age) { }  
greet('John', 25);  
  
function greet(name = 'Hi') { }
```

cost is a **parameter** — temporary variable inside the function.

2000 is the **argument** — value passed in, caught by **cost**.

Multiple params → separate with commas.

Args match params **in order** → name='John', age=25.

Default param — used when no argument is passed.

Return Statement

Gets a value **out** of a function. **Stops execution immediately** when hit.

```
return 5;
```

Outputs **5** from the function and stops immediately.

```
return mathCalculation;
```

Outputs the value stored in a variable.

```
let tax = calcTax();
```

Captures the returned value into a variable.

```
return;
```

Returns **undefined** — exits early (e.g. inside an if-check).

Variable Scope

Scope = where a variable can be accessed.

Local Scope

Variable inside **{ }** — only accessible within those braces.

Global Scope

Variable outside **{ }** — accessible anywhere in the file.

Naming Conflict

Same name is fine **only if** they are in two different local scopes.

Hoisting

JS reads the whole file first and moves function declarations to the top before running.

```
playGame(); // works!
```

Called **before** it is defined — still works.

```
function playGame() { ... }
```

⚠ Only standard functions hoist — not arrow functions.